

mysql基础

一、概念

1.什么是数据库？

数据库是“按照数据结构来组织、存储和管理数据的仓库”。MySQL是一个关系型数据库管理系统，由瑞典MySQL AB 公司开发，属于 Oracle 旗下产品，常用版本mysql5.7。

2.数据库能干嘛？

数据库可以存储数据，账号，密码，存款等数据，常见的数据库有oracle，mysql，mariadb，redis，db2等。

3.MySQL版本说明

Alpha版本：

是内部测试版，一般不向外发布，会有很多Bug 一般只有测试人员使用

Beta版本：

功能开发完和所有测试之后的产品，不会存在较大的功能和性能Bug

RC版本：

生产环境之前的一个小版本，根据Beta版本测试结果打补丁后的版本。

GA版本：

是正式发布的版本

温馨提示：

选择发布六个月以上的GA版本，查看是否有连续BUG的版本，稳定版本一般几个月不会修复重大BUG。

MySQL 8.0新增端口推荐阅读：

<https://dev.mysql.com/doc/mysql-port-reference/en/mysql-ports-reference-tables.html#mysql-client-server-ports>

4.mysql特点

1. MySQL 是开源的，不需要支付额外的费用
2. 支持大型系统，是可以处理拥有上千万条记录的大型数据库
3. 支持多线程，充分利用 CPU 资源
4. 使用标准的 SQL 数据语言形式
5. 跨平台，支持多个操作系统，例如： windows 、 MacOS 、 Linux 等
6. 支持大型系统，是可以处理拥有上千万条记录的大型数据库

二、mysql安装

mysql可以跨平台，支持多个操作系统，例如： Windows 、 MacOS 、 Linux 等，本次安装使用Linux centos的系统版本为mysql5.7。

在 Linux 上安装 MySQL 后，通常会在系统中生成以下文件和目录：

1. 配置文件目录: MySQL 的配置文件通常存储在 `/etc/mysql/` 或 `/etc/my.cnf` 目录下。主要的配置文件是 `my.cnf`。
2. 数据目录: MySQL 数据库的数据文件通常存储在 `/var/lib/mysql/` 目录下。这个目录包含了数据库的实际数据文件, 如表数据、索引等。
3. 日志文件目录: MySQL 的日志文件通常存储在 `/var/log/mysql/` 目录下。常见的日志文件包括错误日志、查询日志等。
4. 临时文件目录: MySQL 的临时文件通常存储在 `/tmp/` 目录下。这些临时文件用于存储临时数据和操作。
5. 安装目录: MySQL 的安装目录通常在 `/usr/bin/` 或 `/usr/local/mysql/` 目录下。这个目录包含了 MySQL 的可执行文件和库文件。
6. Socket 文件目录: MySQL 的 socket 文件通常存储在 `/var/run/mysqld/` 目录下。这个 socket 文件用于 MySQL 与客户端程序之间的通信。

(一) yum 或者 rpm

1.配置yum仓库, 使用yum安装

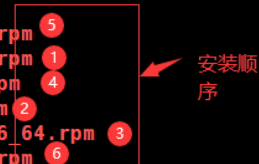
```
#mysql yum仓库配置
yum -y install wget && wget https://dev.mysql.com/get/mysql80-community-release-el7-7.noarch.rpm
yum -y install mysql80-community-release-el7-7.noarch.rpm
#开启具体版本
vim /etc/yum.repos.d/mysql-community.repo
#刷新一下缓存
yum makecache
#查看一下可以安装版本
yum list |grep mysql
#安装
yum -y install mysql-community* --skip-broken
```

2.下载rpm包进行安装

适合没有网络的时候安装

```
#或者自己官网下载rpm包进行安装下载地址
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-server-5.7.36-1.el7.x86_64.rpm
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-client-5.7.36-1.el7.x86_64.rpm
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-common-5.7.36-1.el7.x86_64.rpm
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-libs-5.7.36-1.el7.x86_64.rpm
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-libs-compat-5.7.36-1.el7.x86_64.rpm
wget http://mirrors.ustc.edu.cn/mysql-ftp/Downloads/MYSQL-5.7/mysql-community-libs-compat-5.7.36-1.el7.x86_64.rpm
```

```
[root@localhost /opt/mysql]# ll
总用量 208M
-rw-r--r-- 1 root root 26M 9月 11 2023 mysql-community-client-5.7.36-1.el7.x86_64.rpm
-rw-r--r-- 1 root root 311K 9月 11 2023 mysql-community-common-5.7.36-1.el7.x86_64.rpm
-rw-r--r-- 1 root root 4.0M 9月 11 2023 mysql-community-devel-5.7.36-1.el7.x86_64.rpm
-rw-r--r-- 1 root root 2.4M 9月 11 2023 mysql-community-libs-5.7.36-1.el7.x86_64.rpm
-rw-r--r-- 1 root root 1.3M 9月 11 2023 mysql-community-libs-compat-5.7.36-1.el7.x86_64.rpm
-rw-r--r-- 1 root root 174M 9月 11 2023 mysql-community-server-5.7.36-1.el7.x86_64.rpm
```



3.安装MySQL

第一种方法:

```
yum -y install mysql-community*
```

```
[root@localhost /opt/mysql]# yum -y install mysql-community*
```

正常安装完成

```
已安装:
mysql-community-client.x86_64 0:5.7.36-1.el7
mysql-community-devel.x86_64 0:5.7.36-1.el7
mysql-community-libs-compat.x86_64 0:5.7.36-1.el7
mysql-community-common.x86_64 0:5.7.36-1.el7
mysql-community-libs.x86_64 0:5.7.36-1.el7
mysql-community-server.x86_64 0:5.7.36-1.el7
```

第二种方法:

```
rpm -ivh 包名
```

```
[root@localhost /opt/mysql]# rpm -ivh mysql-community-common-5.7.36-1.el7.x86_64.rpm
```

4.启动mysql

```
#设置开机自启并立即启动
systemctl enable mysqld && systemctl start mysqld
#查看mysql状态
systemctl status mysqld
```

5.修改密码

刚装完的数据库会产生一个临时密码，我们需要使用临时密码将密码修改成自己需要的密码

```
#查看临时密码
grep 'password' /var/log/mysqld.log
#修改mysql密码
mysqladmin -uroot -p'临时密码' password '需要修改的密码'
mysqladmin -uroot -p'o?=RfHasu7Cg' password 'Admin123.'
```

```
[root@localhost /opt/mysql]# systemctl enable mysqld && systemctl start mysqld
[root@localhost /opt/mysql]#
[root@localhost /opt/mysql]# grep 'password' /var/log/mysqld.log
2024-04-01T08:21:11.442017Z 1 [Note] A temporary password is generated for root@localhost: o?=RfHasu7Cg
[root@localhost /opt/mysql]# mysqladmin -uroot -p'o?=RfHasu7Cg' password 'Admin123.'
mysqladmin: [Warning] Using a password on the command line interface can be insecure.
Warning: Since password will be sent to server in plain text, use ssl connection to ensure password safety.
[root@localhost /opt/mysql]#
```

6.登录mysql

```
mysql -h ip -P端口 -u用户名 -p密码
```

选项参数:

- h ip 如果是在本机登录可以不用写默认即可, 如果登录其他机器则使用其他机器ip。
- P(大写) mysql在没有修改端口时默认端口为3306。
- u mysql默认管理员用户为root。
- p(小写) 使用自己的真实修改的密码。

```
mysql -h 127.0.0.1 -P3306 -uroot -p'Admin123.'
```

```
[root@localhost /opt/mysql]# mysql -h 127.0.0.1 -P3306 -uroot -p'Admin123.'
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.36 MySQL Community Server (GPL)

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

7.简单演示基本使用

查看数据库中的库:

```
show databases;
```

--基本库讲解:

information_schema 保存服务器中的基本信息, 如数据库名称权限等。

mysql 保存数据库运行时的信息, 如数据库文件夹, 字符集等。

performance_schema 用于监控mysql各类指标。

sys 也是用于存储系统指标。

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.01 sec)

mysql> █
```

创建一个自己的数据库:

```
create database dbtest;
--进入到数据库
use 库名
use dbtest;
--在当前库中创建一个表
create table test(id int,name verchat(50));
--查看当前库中的表
show tables;
```

(二) 源码包安装

源码包可以自定义设置，但是操作比较复杂

1.安装依赖环境

```
yum install -y cmake make gcc gcc-c++ bison ncurses ncurses-devel openssl-devel
#boost_1_59_0.tar.gz依赖包下载
wget https://sourceforge.net/projects/boost/files/boost/1.59.0/boost_1_59_0.tar.gz/download
```

2.准备安装

```
#下载mysql源码包
wget http://dev.mysql.com/get/Downloads/MySQL-5.7/mysql-5.7.34.tar.gz
#创建组
groupadd mysql
#创建一个账号，属于mysql组，不允许登录系统
useradd -r -g mysql -s /bin/false mysql
#解压mysql包
tar -zxvf mysql-5.7.34.tar.gz
#进入解压好的包
cd mysql-5.7.34
#把boost移动到这里面
cp ../boost_1_59_0.tar.gz .
#解压boost
tar -zxvf boost_1_59_0.tar.gz
#创建mysql存放目录
mkdir -p /mysql5.7/etc /mysql5.7/data /mysql5.7/man /mysql5.7/tmp /mysql5.7/logs
```

3.配置

```
#执行以下配置
[root@f2cfruryo10dalfq mysql-5.7.34]# cmake . \
-DWITH_BOOST=boost_1_59_0/ \
-DCMAKE_INSTALL_PREFIX=/mysql5.7/ \
-DSYSCONFDIR=/mysql5.7/etc/ \
-DMYSQL_DATADIR=/mysql5.7/data/ \
-DINSTALL_MANDIR=/mysql5.7/man/ \
-DMYSQL_TCP_PORT=3306 \
-DMYSQL_UNIX_ADDR=/mysql5.7/tmp/mysql.sock \
-DDEFAULT_CHARSET=utf8 \
-DEXTRA_CHARSETS=all \
-DDEFAULT_COLLATION=utf8_general_ci \
```

```

-DWITH_READLINE=1 \
-DWITH_SSL=system \
-DWITH_EMBEDDED_SERVER=1 \
-DENABLED_LOCAL_INFILE=1 \
-DWITH_INNOBASE_STORAGE_ENGINE=1
#参数说明
cmake #编译之前先配置
-DWITH_BOOST #
-DCMAKE_INSTALL_PREFIX #编译安装的位置在哪里
-DSYSCONFDIR #配置文件目录
-DMYSQL_DATADIR #数据目录位置
-DINSTALL_MANDIR #
-DMYSQL_TCP_PORT #mysql端口
-DMYSQL_UNIX_ADDR #
-DDEFAULT_CHARSET #字符集设置
-DEXTRA_CHARSETS #
-DDEFAULT_COLLATION #

```

4.安装

```

#编译，这个过程需要等很久
make && make install

```

5.初始化

```

#进入mysql目录
cd /mysql5.7
#创建一个目录
mkdir mysql-files
#修改mysql5.7目录的属主和属组
chown -R mysql:mysql /mysql5.7
#初始化它用户使用mysql，基础目录/mysql5.7,数据目录放在/mysql5.7/data里，执行后会生成一个临时密码
/mysql5.7/bin/mysqld --initialize --user=mysql --basedir=/mysql5.7 --datadir=/mysql5.7/data
#对数据目录进行加密5q7=&KqUA6yA
/mysql5.7/bin/mysql_ssl_rsa_setup --datadir=/mysql5.7/data
#创建错误日志，添加到MySQL组
touch /mysql5.7/logs/mysql_error.log
chown -R mysql:mysql /mysql5.7/logs/mysql_error.log
#建立mysql配置文件,先备份mv /etc/my.cnf /etc/my.cnfbak
cat >> /mysql5.7/etc/my.cnf << EOF
[client]
#客户端默认字符集
default-character-set=utf8
[mysqld]
#服务器默认字符集
character-set-server=utf8
collation-server=utf8_general_ci
#服务器ID，在集群中每个节点必须有唯一值
server-id=1
#指定mysql安装目录
basedir=/mysql5.7
#指定数据目录

```

```
datadir=/mysql5.7/data
#开启通用查询日志 不建议开
general_log=OFF
general_log_file=/mysql5.7/logs/mysql_general.log
#开启错误日志（此文件需要先创建并属组给mysql）
log-error=/mysql5.7/logs/mysql_error.log
#开启二进制日志（以mysql-log开头）
log-bin=/mysql5.7/logs/mysql-log
#二进制日志自动删除/过期的天数。默认值为0,表示“没有自动删除”
expire_logs-days=10
EOF
```

6.设置开机自启

```
#复制mysql.server到开机启动目录
cp /mysql5.7/support-files/mysql.server /etc/init.d/mysqld
#添加mysql服务
chkconfig --add mysqld
#开机自启mysql服务
chkconfig mysqld on
#启动mysql
systemctl start mysqld
#修改密码
/mysql5.7/bin/mysqladmin -uroot -p'原临时密码' password 'admin'
#登录mysql
/mysql5.7/bin/mysql -uroot -p'admin'
#将mysql创建快捷方式创建后可以直接使用mysql登录
ln -s /mysql5.7/bin/mysql /usr/bin/mysql
```

三、图形化管理工具

MySQL图形化管理工具极大地方便了数据库的操作与管理，常用的图形化管理工具有：MySQL Workbench、phpMyAdmin、Navicat Premium、MySQLDumper、SQLyog、dbeaver、MySQL ODBC Connector。

1.MySQL Workbench

MySQL官方提供的图形化管理工具MySQL Workbench完全支持MySQL 5.0以上的版本。MySQL Workbench分为社区版和商业版，社区版完全免费，而商业版则是按年收费。MySQL Workbench 为数据库管理员、程序开发者和系统规划师提供可视化设计、模型建立、以及数据库管理功能。它包含了用于创建复杂的数据建模ER模型，正向和逆向数据库工程，也可以用于执行通常需要花费大量时间的、难以变更和管理的文档任务。下载地址：<http://dev.mysql.com/downloads/workbench/>。

2.Navicat

Navicat MySQL是一个强大的MySQL数据库服务器管理和开发工具。它可以与任何3.21或以上版本的MySQL一起工作，支持触发器、存储过程、函数、事件、视图、管理用户等，对于新手来说易学易用。其精心设计的图形用户界面（GUI）可以让用户用一种安全简便的方式来快速方便地创建、组织、访问和共享信息。Navicat支持中文，有免费版提供。下载地址：<http://www.navicat.com/>。

3.SQLyog

SQLyog 是业界著名的 Webyog 公司出品的一款简洁高效、功能强大的图形化 MySQL 数据库管理工具。这款工具是使用C++语言开发的。该工具可以方便地创建数据库、表、视图和索引等，还可以方便地进行插入、更新和删除等操作，同时可以方便地进行数据库、数据表的备份和还原。该工具不仅可以通过SQL文件进行大量文件的导入和导出，还可以导入和导出XML、HTML和CSV等多种格式的数据。下载地址：<http://www.webyog.com/>。

四、基础概念

2.运算符

mysql中运算符一般用做查询时候使用，在某些情况下需要进行使用，运算符种类：算术运算符，比较运算符，逻辑运算符，位运算符。

2.1算数运算符

算术运算符主要用于数学运算，其可以连接运算符前后的两个数值或表达式，对数值或表达式进行加（+）、减（-）、乘（*）、除（/）和取模（%）运算。

运算符	名称	作用	示例
+	加法运算符	计算两个值或表达式的和	select 1+1
-	减法运算符	计算两个值或表达式的差	select 3-2
*	乘法运算符	计算两个值或表达式的积	select 2*3
/	除法运算符	计算两个值或表达式的商	select 4/2
%	求余运算符	计算两个值或表达式的余	select 8%2

2.2比较运算符

比较运算符用来对表达式左边的操作数和右边的操作数进行比较，比较的结果为真则返回1，比较的结果为假则返回0，其他情况则返回NULL。以下是常用比较运算符：

=	等于
!=	不等于
<	小于
<=	小于等于
>	大于
>=	大于等于

2.3逻辑运算符

运算符	作用
NOT 或者 !	逻辑非
and 或 &&	逻辑与
or 或	逻辑或
xor	逻辑异或

2.4位运算符

位运算符是在二进制数上进行计算的运算符。位运算符会先将操作数变成二进制数，然后进行位运算，最后将计算结果从二进制变回十进制数。 以下是常见的位运算符

运算符	作用
&	and
	or

3.mysql的字符集

查看字符集

```
SHOW VARIABLES LIKE 'character_%';
```

```
mysql> SHOW VARIABLES LIKE 'character_%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| character_set_client | utf8 |
| character_set_connection | utf8 |
| character_set_database | latin1 |
| character_set_filesystem | binary |
| character_set_results | utf8 |
| character_set_server | latin1 |
| character_set_system | utf8 |
| character_sets_dir | /usr/share/mysqlCharsets/ |
+-----+-----+
8 rows in set (0.00 sec)

mysql> 
```

查看比较规则

```
SHOW VARIABLES LIKE 'collation_%';
```

```
mysql> SHOW VARIABLES LIKE 'collation_%';
+-----+-----+
| Variable_name | Value               |
+-----+-----+
| collation_connection | utf8_general_ci    |
| collation_database   | latin1_swedish_ci  |
| collation_server     | latin1_swedish_ci  |
+-----+-----+
3 rows in set (0.01 sec)

mysql> 
```

修改字符集和比较规则

```
#修改这两个参数需要去mysql配置文件中修改,修改后重启一下数据库即可
vim /etc/my.cnf
character_set_server=utf8mb4
collation_server=utf8mb4_general_ci
```

修改已创建数据库和表的字符集:

```
--修改已经创建的数据库的字符集
alter database 库名 character set 'utf8mb4' collate 'utf8mb4_general_ci';
--修改已创建表的字符集
alter table 表名 convert to character set 'utf8mb4' collate 'utf8mb4_general_ci';
```

4.函数

4.1什么是函数?

函数在计算机中它可以把我们经常使用的代码封装起来,需要的时候直接调用即可。能提高了代码效率,提高可维护性。在SQL中我们也可以使用函数对检索出来的数据进行函数操作。使用这些函数,可以极大地提高用户对数据库的管理效率。函数可以分为内置函数和自定义函数,内置函数系统自带,自定义函数需要我们自己定义。

MySQL提供的内置函数可以分为数值函数、字符串函数、日期和时间函数、流程控制函数、加密与解密函数、获取MySQL信息函数、聚合函数等。本记录只记录部分常用函数

4.2日期时间函数

获取日期、时间:

函数	作用	用法
CURDATE()	返回当前日期,只包含年、月、日	SELECT CURDATE() FROM DUAL;
CURTIME()	返回当前时间,只包含时、分、秒	SELECT CURTIME() FROM DUAL;
NOW()	返回当前系统日期和时间	SELECT NOW() FROM DUAL;

日期与时间戳的转换:

函数	作用	用法
UNIX_TIMESTAMP()	以UNIX时间戳的形式返回当前时间	SELECT UNIX_TIMESTAMP(now());
UNIX_TIMESTAMP(date)	将时间date以UNIX时间戳的形式返回	SELECT UNIX_TIMESTAMP(CURDATE());
FROM_UNIXTIME(timestamp)	将UNIX时间戳的时间转换为普通格式的时间	SELECT FROM_UNIXTIME(timestamp);

4.3流程处理函数

函数	作用	用法
IF(value,value1,value2)	如果value的值为TRUE，返回value1，否则返回value2	SELECT IF(1 > 0,'正确','错误')
IFNULL(value1, value2)	如果value1不为NULL，返回value1，否则返回value2	SELECT IFNULL(null,'Hello Word')
CASE WHEN 条件1 THEN 结果1 WHEN 条件2 THEN 结果2.... [ELSE resultn] END	将UNIX时间戳的时间转换为普通格式的时间	SELECT CASE WHEN 1 > 0 THEN '1 > 0' END

4.4加密解密

函数	作用	用法
MD5(str)	返回字符串str的md5加密后的值，也是一种加密方式。若参数为NULL，则会返回NULL	SELECT md5('123')
SHA(str)	从原明文密码str计算并返回加密后的密码字符串，当参数为NULL时，返回NULL。SHA加密算法比MD5更加安全	SELECT SHA('Tom123')

4.5信息函数

函数	作用	用法
VERSION()	返回当前MySQL的版本号	SELECT VERSION()
CONNECTION_ID()	返回当前MySQL服务器的连接数	SELECT CONNECTION_ID()
DATABASE()	返回MySQL命令行当前所在的数据库	SELECT DATABASE()
USER(), CURRENT_USER(), SYSTEM_USER(), SESSION_USER()	返回当前连接MySQL的用户名，返回结果格式为“主机名@用户名”	SELECT USER(),CURRENT_USER(),SYSTEM_USER(),SESSION_USER()

4.6聚合函数

函数	作用	用法
COUNT()	统计总数等	SELECT COUNT(*) from table;
AVG()	统计平均值	SELECT avg(age) from table;
SUM()	求和	SELECT sum(wages) from table;
MAX()	最大值	SELECT MAX(xx) from table;
MIN()	最小值	SELECT MIN(xx) from table;

5.mysql配置文件

```
[client] #客户端设置
port=3306 #服务器监听端口，默认为3306
socket=/usr/local/mysql/tmp/mysql.sock #Unix套接字文件路径，默认/tmp/mysql.sock
default-character-set=utf8mb4
[mysqld] #服务端设置
## 一般配置选项
port=3306 #服务器监听端口，默认为3306
basedir=/usr/local/mysql #MySQL安装根目录，默认/usr/bin/mysql
datadir=/usr/local/mysql/data #MySQL数据文件目录
socket=/usr/local/mysql/tmp/mysql.sock #Unix套接字文件路径，默认/tmp/mysql.sock
pid-file=/usr/local/mysql/tmp/mysql.pid #服务进程pid文件路径
character_set_server=utf8 #默认字符集
default_storage_engine=InnoDB #默认InnoDB存储引擎
user=mysql
## 连接配置选项
max_connections=200 #最大并发连接数
table_open_cache=400 #表打开缓存大小，默认2000
open_files_limit=1000 #打开文件数限制，默认5000
max_connect_errors=200 #最大连接失败数，默认100
back_log=100 #请求连接队列数
connect_timeout=20 #连接超时时间，默认10秒
interactive_timeout=1200 #交互式超时时间，默认28800秒
wait_timeout=600 #非交互超时时间，默认28800秒
net_read_timeout=30 #读取超时时间，默认30秒
net_write_timeout=60 #写入超时时间，默认60秒
max_allowed_packet=8M #最大传输数据字节，默认4M
thread_cache_size=10 #线程缓冲区（池）大小
thread_stack=256K #线程栈大小，32位平台196608、64位平台262144
## 临时内存配置选项
tmpdir=/tmp #临时目录路径
tmp_table_size=64M #临时表大小，默认16M
max_heap_table_size=64M #最大内存表大小，默认16M
sort_buffer_size=1M #排序缓冲区大小，默认256K
## InnoDB配置选项
#innodb_thread_concurrency=0 #InnoDB线程并发数
innodb_io_capacity=200 #IO容量，可用于InnoDB后台任务的每秒I/O操作数（IOPS），
innodb_io_capacity_max=400 #IO最大容量，InnoDB在这种情况下由后台任务执行的最大IOPS数
innodb_lock_wait_timeout=50 #InnoDB引擎锁等待超时时间，默认50（单位：秒）
innodb_buffer_pool_size=512M #InnoDB缓冲池大小，默认128M
```

```

innodb_buffer_pool_instances=4 #InnoDB缓冲池划分区域数
innodb_max_dirty_pages_pct=75 #缓冲池最大允许脏页比例，默认为75
innodb_flush_method=O_DIRECT #日志刷新方法，默认为fdasync
innodb_flush_log_at_trx_commit=2 #事务日志刷新方式，默认为0
transaction_isolation=REPEATABLE-READ #事务隔离级别，默认REPEATABLE-READ
innodb_data_home_dir=/usr/local/mysql/data #表空间文件路径，默认保存在MySQL的datadir中
innodb_data_file_path=ibdata1:128M:autoextend #表空间文件大小
innodb_file_per_table=ON #数据存放系统表空间还是独立表空间
innodb_log_group_home_dir=/usr/local/mysql/data #redoLog文件目录，默认保存在MySQL的datadir中
innodb_log_files_in_group=2 #日志组中的日志文件数，默认为2
innodb_log_file_size=128M #日志文件大小，默认为48MB
innodb_log_buffer_size=32M #日志缓冲区大小，默认为16MB
## MyISAM配置选项
key_buffer_size=32M #索引缓冲区大小，默认8M
read_buffer_size=4M #顺序读缓冲区大小，默认128K
read_rnd_buffer_size=4M #随机读缓冲区大小，默认256K
bulk_insert_buffer_size=8M #块插入缓冲区大小，默认8M
myisam_sort_buffer_size=8M #MyISAM排序缓冲大小，默认8M
#myisam_max_sort_file_size=1G #MyISAM排序最大临时大小
myisam_repair_threads=1 #MyISAM修复线程
skip_external_locking #跳过外部锁定，启用文件锁会影响性能
## 日志配置选项
log_output=FILE #日志输出目标，TABLE（输出到表）、FILE（输出到文件）、NONE（不输出），可选择一个或多个以
逗>号分隔
log_error=/usr/local/mysql/logs/error.log #错误日志存放路径
log_error_verbosity=1 #错误日志过滤，允许的值为1（仅错误），2（错误和警告），3（错误、警告和注释），默认值
为3。
log_timestamps=SYSTEM #错误日志消息格式，日志中显示时间戳的时区，UTC（默认值）和SYSTEM（本地系统时区）
general_log=ON #开启查询日志，一般选择不开启，因为查询日志记录很详细，会增大磁盘IO开销，影响性能
general_log_file=/usr/local/mysql/logs/general.log #通用查询日志存放路径
## 慢查询日志配置选项
slow_query_log=ON #开启慢查询日志
slow_query_log_file=/usr/local/mysql/logs/slowq.log #慢查询日志存放路径
long_query_time=2 #慢查询时间，默认10（单位：秒）
min_examined_row_limit=100 #最小检查行限制，检索的行数必须达到此值才可被记为慢查询
log_slow_admin_statements=ON #记录慢查询管理语句
log_queries_not_using_indexes=ON #记录查询未使用索引语句
log_throttle_queries_not_using_indexes=5 #记录未使用索引速率限制，默认为0不限制
log_slow_slave_statements=ON #记录从库复制的慢查询，作为从库时生效，从库复制中如果有慢查询也将被记录
## 复制配置选项
server-id=1 #MySQL服务唯一标识
log-bin=mysql-bin #开启二进制日志，默认位置是datadir数据目录
log-bin-index=mysql-bin.index #binlog索引文件
binlog_format=MIXED #binlog日志格式，分三种：STATEMENT、ROW或MIXED，MySQL 5.7.7之前默认为
STATEMENT，之后默认为ROW
binlog_cache_size=1M #binlog缓存大小，默认32KB
max_binlog_cache_size=1G #binlog最大缓存大小，推荐最大值为4GB
max_binlog_size=256M #binlog最大文件大小，最小值为4096字节，最大值和默认值为1GB
expire_logs_days=7 #binlog过期天数，默认为0不自动删除
log_slave_updates=ON #binlog级联复制
sync_binlog=1 #binlog同步频率，0为禁用同步（最佳性能，但可能丢失事务），为1开启同步（影响性能，但最安全不会
丢失任何事务），为N操作N次事务后同步1次
relay_log=relay-bin #relaylog文件路径，默认位置是datadir数据目录

```

```

relay_log_index=relay-log.index #relaylog索引文件
max_relay_log_size=256M #relaylog最大文件大小
relay_log_purge=ON #中继日志自动清除，默认值为1（ON）
relay_log_recovery=ON #中继日志自动恢复
auto_increment_offset=1 #自增值偏移量
auto_increment_increment=1 #自增值自增量
slave_net_timeout=60 #从机连接超时时间
replicate-wild-ignore-table=mysql.% #复制时忽略的数据库表，告诉从线程不要复制到与给定通配符模式匹配的表
skip-slave-start #跳过slave启动，slave复制进程不随MySQL启动而启动
## 其他配置选项
#memlock=ON #开启内存锁，此选项生效需系统支持mlockall()调用，将mysqld进程锁定在内存中，防止遇到操作系统导致mysqld交换到磁盘的问题
lower_case_table_names=1 #设置不区分大小写
[mysqldump] #mysqldump数据库备份工具
quick #强制mysqldump从服务器查询取得记录直接输出，而不是取得所有记录后将它们缓存到内存中
max_allowed_packet=16M #最大传输数据字节，使用mysqldump工具备份数据库时，某表过大会导致备份失败，需要增大该值（大于表大小即可）

[myisamchk] #使用myisamchk实用程序可以用来获得有关你的数据库表的统计信息或检查、修复、优化他们
key_buffer_size=32M #索引缓冲区大小
myisam_sort_buffer_size=8M #排序缓冲区大小
read_buffer_size=4M #读取缓冲区大小
write_buffer_size=4M #写入缓冲区大小

```

6.基本配置

```

#客户端设置
[client]

#mysql服务配置
[mysqld]
#服务器监听端口，默认为3306
port=3306
#数据存储位置
datadir=/var/lib/mysql
#Unix套接字文件路径
socket=/var/lib/mysql/mysql.sock
#MySQL服务唯一标识,开启二进制日志，做主从时需要
server-id=1
log-bin=mysql-bin
#不会跟随符号链接
symbolic-links=0
#日志路径
log-error=/var/log/mysqld.log
#服务进程pid文件路径
pid-file=/var/run/mysqld/mysqld.pid
#mysql字符集设置
character_set_server=utf8mb4
#mysql比较规则设置
collation_server=utf8mb4_general_ci

```

五、sql语言

1.sql语言概述

1.1什么是sql语言？

sql语言（结构化查询语言），主要用于存储数据，查询数据，更新数据，管理数据库系统。sql语言由IBM开发。

DDL语句	数据库定义语言：用于定义数据库，表，索引，视图，存储过程，
DML语句	数据库操纵语言：插入数据insert，删除数据delete，更新数据update
DQL语句	数据库查询语言：查询数据select
DCL语句	数据库控制语言：定义数据库、表、字段、用户的访问权限和安全级别

1.2sql语言的基本规则

- 1 SQL 可以写在一行或者多行。为了提高可读性，各子句分行写，必要时使用缩进
- 2 每条命令以 ； 或 \g 或 \G 结束
- 3 关键字不能被缩写也不能分行
- 4 关于标点符号：
 - 必须保证所有的()、单引号、双引号是成对结束的
 - 必须使用英文状态下的半角输入方式
 - 字符串型和日期时间类型的数据可以使用单引号（' '）表示
 - 列的别名，尽量使用双引号（" "），而且不建议省略as

1.3sql语言基本规范

- 1 MySQL 在 windows 环境下是大小写不敏感的
- 2 MySQL 在 Linux 环境下是大小写敏感的
 - 数据库名、表名、表的别名、变量名是严格区分大小写的
 - 关键字、函数名、列名(或字段名)、列的别名(字段的别名) 是忽略大小写的。
- 3 推荐采用统一的书写规范：
 - 数据库名、表名、表别名、字段名、字段别名等都小写
 - SQL 关键字、函数名、绑定变量等都大写

1.4sql注释

- 1 单行注释：#注释文字(MySQL特有的方式)
- 2 单行注释：-- 注释文字(--后面必须包含一个空格。)
- 3 多行注释：/* 注释文字 */

1.5命名规则

- 1 数据库、表名不得超过30个字符，变量名限制为29个
- 2 必须只能包含 A-Z，a-z，0-9，_共63个字符
- 3 数据库名、表名、字段名等对象名中间不要包含空格
- 4 同一个MySQL软件中，数据库不能同名；同一个库中，表不能重名；同一个表中，字段不能重名
- 5 必须保证你的字段没有和保留字、数据库系统或常用方法冲突。如果坚持使用，请在SQL语句中使用（着重号）引起来
- 6 保持字段名和类型的一致性，在命名字段并为其指定数据类型的时候一定要保证一致性。假如数据类型在一个表里是整数，那在另一个表里可就别变成字符型了

2.DDL

DDL数据库定义语言:语句关键字包括 CREATE,DROP , ALTER,RENAME,TRUNCATE 等。是用于定义数据库对象（如表、索引、视图等）的一组 SQL 命令。

2.1创建和管理数据库

创建数据库：

```
--方式1：创建数据库
CREATE DATABASE 数据库名；
--方式2：创建数据库并指定字符集
CREATE DATABASE 数据库名 CHARACTER SET 字符集；
--方式3：判断数据库是否已经存在，不存在则创建数据库（推荐）
CREATE DATABASE IF NOT EXISTS 数据库名；
```

使用数据库：

```
--查看当前所有的数据库
SHOW DATABASES；#有一个S，代表多个数据库
--查看当前正在使用的数据库
SELECT DATABASE()；
--查看指定库下所有的表
SHOW TABLES FROM 数据库名；
--查看数据库的创建信息
SHOW CREATE DATABASE 数据库名；
--使用/切换数据库
USE 数据库名；
```

修改数据库：

```
--更改数据库字符集
ALTER DATABASE 数据库名 CHARACTER SET 字符集；#比如：gbk、utf8等
```

删除数据库：

```
--方式1：删除指定的数据库
DROP DATABASE 数据库名；
--方式2：删除指定的数据库（推荐）
DROP DATABASE IF EXISTS 数据库名；
```

2.2创建和管理表

数据类型：一般在创建表的时候给每个字段设置一个数据类型，以下是常见的数据类型：

数据类型	描述
INT	从-2^31到2^31-1的整型数据。存储大小为 4个字节
CHAR(size)	定长字符数据。若未指定，默认为1个字符，最大长度255
VARCHAR(size)	可变长字符数据，根据字符串实际长度保存，必须指定长度
FLOAT(M,D)	单精度，占用4个字节，M=整数位+小数位，D=小数位。D<=M<=255,0<=D<=30，默认M+D<=6
DOUBLE(M,D)	双精度，占用8个字节，D<=M<=255,0<=D<=30，默认M+D<=15
DECIMAL(M,D)	高精度小数，占用M+2个字节，D<=M<=65，0<=D<=30，最大取值范围与DOUBLE相同
DATE	日期型数据，格式'YYYY-MM-DD'
BLOB	二进制形式的长文本数据，最大可达4G
TEXT	长文本数据，最大可达4G

创建表：

```
--语法
CREATE TABLE [IF NOT EXISTS] 表名(
  字段1, 数据类型 [约束条件] [默认值],
  字段2, 数据类型 [约束条件] [默认值],
  字段3, 数据类型 [约束条件] [默认值],
  .....
  [表约束条件]
);
--创建表示例
CREATE TABLE table1 (t_id INT,t_name VARCHAR(20),salary DOUBLE,birthday DATE);
--查看表结构
DESC table1;
--查看数据表结构
SHOW CREATE TABLE table1;
--重命名表名
RENAME TABLE table1 TO mytable;
```

修改表：

```

--追加一个列
ALTER TABLE 表名 ADD 字段名 字段类型;
ALTER TABLE table1 ADD job_id varchar(15);
--修改一个列
ALTER TABLE 表名 MODIFY 字段名1 字段类型;
ALTER TABLE table1 MODIFY t_name VARCHAR(30);
--重命名一个列
ALTER TABLE 表名 CHANGE 列名 新列名 新数据类型;
ALTER TABLE table1 CHANGE t_name t1_name varchar(15);
--删除一个列
ALTER TABLE 表名 DROP COLUMN 字段名
ALTER TABLE table1 DROP COLUMN job_id;

```

删除表:

```

--此操作不能回滚, 谨慎操作
--方法一
DROP TABLE [IF EXISTS] table1;
--方法二
DROP TABLE table1;

```

清空表:

```

--此操作不能回滚, 谨慎操作
TRUNCATE TABLE 表名;
TRUNCATE TABLE table1;
--可以回滚
DELETE FROM 表名;
DELETE FROM table1;

```

3.DML

数据库操纵语言: 插入数据insert, 删除数据delete, 更新数据update

3.1插入数据insert

```

--方式1: 为表的所有字段按默认顺序插入数据
INSERT INTO 表名 VALUES (value1,value2,...);
INSERT INTO 表名 (字段1,字段2,...) VALUES (value1,value2,...);
insert into mytable value (1,'hhh',3000,'2000-01-01');
--方式2: 指定字段添加
insert into mytable (t_id,t_name) value (2,'aaa');
--方式3: 同时添加多条
insert into mytable value (3,'bbb',3000,'2000-01-01'),(4,'ccc',3000,'2000-01-01');
insert into mytable (t_id,t_name) value (5,'ddd'),(6,'eee');

```

3.2删除数据delete

```
--删除数据
delete from 表名 where 条件;
delete from mytable where t_id =6;
--如果省略 WHERE 子句, 则表中的全部数据将被删除
delete from 表名;
```

3.3更新数据update

```
--修改表数据
update 表名 set 修改内容 where 修改条件;
update mytable set t_name='fff' where t_id=2;
```

4.DQL

DQL主要是select, 用于数据库查询数据, 使用频率最多。

4.1select最基本的语法

```
--基本语法
select * from 表名;或者select 字段一,字段二 from 表名;
select * from table;
参数讲解:
    SELECT 标识选择哪些列
    *    列出所有字段
    FROM 标识从哪个表中选择
--最基本的查询
select 1+1,1*2 from dual; --dual属于伪表
```

4.2列的别名

```
--查询一些数据时, 我们想重新给字段设置一个名字, 我们就可以使用别名。别名可以紧跟列名, 也可以在列名和别名之间加入关键字AS, 别名使用双引号, 以便在别名中包含空格或特殊的字符并区分大小写
--使用as用法
select name as 姓名,age as 年龄 from table;
--不使用as
select name 姓名,age 年龄 from table;
```

4.3去除重复项

```
--默认情况下, 查询会返回全部行, 所以很有可能出现很多重复项比如通话记录中电话号码就会很多重复, 可以使用关键字DISTINCT去除重复项
SELECT DISTINCT phone FROM table;
```

4.4查询常数

```
--查询有些数据时, 想在查询结果添加一个字段就需要使用查询常数, 比如每一行都加一个性别
select name,'男' from table;
```

4.5where条件查询

--查询数据时，需要筛选过滤一些数据就可以使用where条件语句。比如查找年龄18岁以上的人员
select * from table where age>18;

4.6ORDER BY 排序

--查询数据时，需要对查询的数据进行排序可以使用ORDER BY进行排序,ORDER BY 子句在SELECT语句的结尾
--排序规则:使用 ORDER BY 子句排序,ASC (升序) ,DESC (降序) 默认排序是asc。
--单列排序
select * from table order by id;
select * from table order by id desc;
--多列排序
select * from table order by id,age desc;

4.7limit分页

--查询数据时，不想加载所有数据我们可以使用limit进行分页
--查询前10行数据
select * from table limit 10;
--查询20到30行的数据
select * from table limit 20,30;

4.8多表查询

--在正常环境中我们不会把所有数据放在同一张表中，会存放在多张表里面比如学生表和教师表，但是又想查询每个学生对应的老师就需要用到多表查询，多表查询一般两个表都需要有相互关联的字段
select * from student s,teacher t where s.tid=t.tid;

4.9GROUP BY分组

--在查询时需要对一个字段进行分组可以使用group by，比如查询每个部门平均工资
SELECT departmentid, AVG(salary) FROM employees GROUP BY departmentid;

4.10子查询

--子查询是指在 SQL 查询语句中嵌套使用的查询语句。子查询也被称为内部查询或嵌套查询。子查询可以出现在 SELECT、INSERT、UPDATE 或 DELETE 语句中的 WHERE 子句中，用于提供更复杂的查询逻辑和过滤条件。
select * from table1 where table1.id in (select * from table2);

5.DCL

数据库控制语言：定义数据库、表、字段、用户的访问权限和安全级别

1.查看权限

--查看权限列表
SHOW PRIVILEGES;
--查看当前用户的权限
show grants;
--查看某个用户的权限
SHOW GRANTS FOR '用户名';

2.权限列表

权限分布	可设置的权限
表权限	SELECT、INSERT、UPDATE、DELETE等
列权限	SELECT、INSERT、UPDATE等
过程权限	对存储过程的执行权限

3.赋予权限

```
--语法
GRANT 权限1, 权限2,... on 数据库名.表名 to '用户名'@'用户地址' IDENTIFIED BY '密码';
GRANT select,update ON mydb.mytable TO 'test'@'%';
--赋予所有的权限
GRANT ALL PRIVILEGES ON *.* TO 'test'@'%';
--注意：给其他用户赋予了所有权限之后，其他用户是不可以对另外的用户进行赋予权限的，这就是和root的区别，如果需要给普通用户赋予权限的需求可以使用WITH GRANT OPTION
--语法
GRANT 权限列表 ON 库名.表名 TO '用户名'@'客户端主机ip' IDENTIFIED BY '密码' WITH GRANT OPTION;
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY 'Admin123.' WITH GRANT OPTION;
--最终需要刷新
FLUSH PRIVILEGES;
```

4.收回权限

```
--语法
REVOKE 权限1, 权限2,... ON 数据库名.表名 TO '用户名'@'用户地址' IDENTIFIED BY '密码';
REVOKE ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY 'Admin123.' WITH GRANT OPTION;
--最终需要刷新
FLUSH PRIVILEGES;
```

六、用户管理

MySQL的用户是用来管理数据库和表，mysql把用户分成普通用户和root用户，root用户是超级管理员拥有所有权限。

1.查看，创建，修改，删除用户

1.1查看数据库中的用户

```
SELECT user,host,authentication_string from mysql.user;
```

1.2创建用户

```
--语法
CREATE USER 用户名 IDENTIFIED BY 密码;
--示例
create user 'test' identified by 'test';
CREATE USER 'test'@'%' IDENTIFIED BY 'test';
```

1.3修改用户

```
--方法一
update mysql.user set host='localhost' where user='test'
--方法二
ALTER USER 'test'@'localhost' IDENTIFIED BY 'test';
--修改后需要进行刷新
flush privileges;
```

1.4删除用户

```
--方式一，删除所有的
DROP USER 用户名;
DROP USER 'test';
--方式二，指定host删除
DROP USER 用户名@host;
DROP USER 'test'@'localhost';
```

2.修改密码

```
ALTER USER 用户名 IDENTIFIED BY 密码;
ALTER USER 'test' IDENTIFIED BY 'Admin';
```

3.密码过期策略

```
--设置密码立即过期
alter user 用户名 password expire;
--设置过期时间
alter user 用户名 password expire interval 90 day;
```

七、约束

1.什么是约束

约束是表级的强制规定。可以在创建表时规定约束或者在表创建之后通过 ALTER TABLE 语句规定约束。它是防止数据库中存在不符合语义规定的数据和防止因错误信息的输入输出造成无效操作或错误信息而提出的。

约束的分类：

单列约束	每个约束只约束一列
多列约束	每个约束可约束多列数据
列级约束	只能作用在一个列上，跟在列的定义后面
表级约束	可以作用在多个列上，不与列一起，而是单独定义

根据约束起的作用，约束可分为：

NOT NULL 非空约束, 规定某个字段不能为空
UNIQUE 唯一约束, 规定某个字段在整个表中是唯一的
PRIMARY KEY 主键(非空且唯一)约束
FOREIGN KEY 外键约束
CHECK 检查约束
DEFAULT 默认值约束

2.非空约束

特点: 限定某个字段/某列的值不允许为空,非空约束只能出现在表对象的列上, 只能某个列单独限定非空, 不能组合非空,空字符串"不等于NULL, 0也不等于NULL。

2.1创建非空约束

```
--建表时创建约束
CREATE TABLE 表名称(
  字段名 数据类型,
  字段名 数据类型 NOT NULL,
  字段名 数据类型 NOT NULL
);
CREATE TABLE student(
  id int,
  name varchar(20) not null
);
--表创建好后添加
alter table 表名称 modify 字段名 数据类型 not null;
alter table student modify id int not null;
```

2.2删除非空约束

```
alter table 表名称 modify 字段名 数据类型 NULL;
ALTER TABLE student MODIFY name VARCHAR(30) NULL;
或
alter table 表名称 modify 字段名 数据类型;
ALTER TABLE student MODIFY name VARCHAR(30);
```

3.唯一性约束

特点: 用来限制某个字段/某列的值不能重复。同一个表可以有多个唯一约束, 唯一约束可以是某一个列的值唯一, 也可以多个列组合的值唯一。MySQL会给唯一约束的列上默认创建一个唯一索引。

3.1添加唯一约束

```
--方法一, 创建表时添加
create table 表名称(
  字段名 数据类型,
  字段名 数据类型 unique,
  字段名 数据类型 unique key,
  字段名 数据类型
);
CREATE TABLE table(
  id INT UNIQUE,
```

```

name VARCHAR(100) UNIQUE,
tel cahr(11) unique key
);
--方法二，创建表时添加
create table 表名称(
字段名 数据类型,
字段名 数据类型,
字段名 数据类型,
[constraint 约束名] unique key(字段名)
);
CREATE TABLE USER(
id INT NOT NULL,
NAME VARCHAR(25),
PASSWORD VARCHAR(16),
-- 使用表级约束语法
CONSTRAINT uk_name_pwd UNIQUE(NAME,PASSWORD)
);
--建表后指定唯一键约束
alter table 表名称 add unique key(字段列表);
alter table 表名称 modify 字段名 字段类型 unique;

```

3.2删除唯一约束

```

--查看都有哪些约束
SELECT * FROM information_schema.table_constraints WHERE table_name = '表名';
ALTER TABLE USER DROP INDEX uk_name_pwd;

```

4.PRIMARY KEY约束

特点：用来唯一标识表中的一行记录。键约束相当于唯一约束+非空约束的组合，主键约束列不允许重复，也不允许出现空值。一个表最多只能有一个主键约束，建立主键约束可以在列级别创建，也可以在表级别上创建。

4.1添加主键约束

```

--方法一，创建表时添加
create table 表名称(
字段名 数据类型 primary key, #列级模式
字段名 数据类型,
字段名 数据类型
);
create table table1(
id int primary key,
name varchar(20)
);
--方法二，创建表时添加
create table 表名称(
字段名 数据类型,
字段名 数据类型,
字段名 数据类型,
[constraint 约束名] primary key(字段名) #表级模式
);
CREATE TABLE table2(
id INT NOT NULL,

```



```
NAME VARCHAR(20),  
pwd VARCHAR(15),  
CONSTRAINT emp7_pk PRIMARY KEY(NAME,pwd)  
);
```

4.2删除主键约束

```
alter table 表名称 drop primary key;
```

5.自增列: AUTO_INCREMENT

特点: 某个字段的值自增。一个表最多只能有一个自增长列, 当需要产生唯一标识符或顺序值时, 可设置自增长

5.1创建自增约束

```
--方法一, 创建表时添加  
create table 表名称(  
字段名 数据类型 primary key auto_increment,  
字段名 数据类型 unique key not null,  
字段名 数据类型 unique key,  
字段名 数据类型 not null default 默认值,  
);  
--方法二, 创建表时添加  
create table 表名称(  
字段名 数据类型 default 默认值 ,  
字段名 数据类型 unique key auto_increment,  
字段名 数据类型 not null default 默认值,,  
primary key(字段名)  
);  
--建表后  
alter table 表名称 modify 字段名 数据类型 auto_increment;
```

5.2删除自增约束

```
alter table 表名称 modify 字段名 数据类型;
```

八、视图

1.什么是视图

视图是一种 虚拟表, 本身是 不具有数据 的, 占用很少的内存空间, 它是 SQL 中的一个重要概念。视图建立在已有表的基础上, 视图赖以建立的这些表称为基表。

2.创建视图

```
CREATE VIEW 视图名称 AS 查询语句;  
CREATE VIEW table_view AS SELECT id,name,salary FROM table;
```

3.查看视图

```
--查看数据库的表对象、视图对象
SHOW TABLES;
--查看视图的结构
DESC / DESCRIBE 视图名称;
```

4.修改视图

```
ALTER VIEW 视图名称 AS 查询语句
```

5.删除视图

```
--删除视图只是删除视图的定义，并不会删除基表的数据
DROP VIEW IF EXISTS 视图名称;
```

九、存储过程

1.什么是存储过程

MySQL的存储过程是一组预先编译好的SQL语句集合，类似于程序中的函数，可以在MySQL服务器上存储和执行。存储过程可以接受参数、执行SQL查询、控制流程、进行逻辑处理，并返回结果。存储过程可以被多次调用，提高了SQL代码的重用性和可维护性。

好处：

1、简化操作，提高了sql语句的重用性，减少了开发程序员的压力 2、减少操作过程中的失误，提高效率 3、减少网络传输量（客户端不需要把所有的 SQL 语句通过网络发给服务器） 4、减少了 SQL 语句暴露在网上的风险，也提高了数据查询的安全性

2.创建存储过程

```
--语法
CREATE PROCEDURE 存储过程名(IN|OUT|INOUT 参数名 参数类型,...)
[characteristics ...]
BEGIN
存储过程体
END
--示例
CREATE PROCEDURE select_mytable()
BEGIN
SELECT * FROM mytable;
END
```

3.存储过程的调用

```
CALL 存储过程名(实参列表);
call select_mytable();
```

4.删除存储过程

```
DROP PROCEDURE 存储过程名
DROP PROCEDURE select_mytable;
```

十、触发器

1.什么是触发器

MySQL触发器是一种特殊的数据库对象，它与表相关联，并在表上执行定义的操作（例如插入、更新、删除）时自动触发。触发器是一种用于实现数据库约束、自动化业务逻辑和数据一致性的机制。

2.创建触发器

```
--语法结构
CREATE TRIGGER 触发器名称
{BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON 表名
FOR EACH ROW
--说明
表名：表示触发器监控的对象。
    BEFORE|AFTER：表示触发的时间。BEFORE 表示在事件之前触发；AFTER 表示在事件之后触发。
    INSERT|UPDATE|DELETE：表示触发的事件。
    INSERT 表示插入记录时触发；
    UPDATE 表示更新记录时触发；
    DELETE 表示删除记录时触发。
CREATE TRIGGER 触发器名称
{BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON 表名
FOR EACH ROW
```

示例：

```
--创建两个表
CREATE TABLE mytb (
id INT PRIMARY KEY AUTO_INCREMENT,
user_name VARCHAR(30)
);
CREATE TABLE mytb_log (
id INT PRIMARY KEY AUTO_INCREMENT,
log VARCHAR(30)
);
--创建触发器
CREATE TRIGGER mytergger
AFTER INSERT ON mytb
FOR EACH ROW
BEGIN
    INSERT INTO mytb_log (log) VALUES ('mytb表插入了新的数据');
END
--验证
insert into mytb (user_name) value ('hhh')
select * from mytb_log
```

3.查看触发器

```
--方式1: 查看当前数据库的所有触发器的定义
SHOW TRIGGERS\G
--方式2: 查看当前数据库中某个触发器的定义
SHOW CREATE TRIGGER 触发器名
--方式3: 从系统库information_schema的TRIGGERS表中查询“salary_check_trigger”触发器的信息。
SELECT * FROM information_schema.TRIGGERS;
```

4.删除触发器

```
DROP TRIGGER IF EXISTS 触发器名称;
```

十一、日志

1.日志分类

```
Error Log    --错误日志: 启动, 停止, 关闭失败报错。
General query log --所有查询都记在这里, 默认关闭
Binary log   --二进制日志: 实现备份, 增量备份。除了select都记
Relay log    --中继日志: 读取主服务器的binlog, 在本地回放, 保持一致
slow log     --慢查询日志, 指导调优。
DDL log      --定义语句的日志
```

十二、备份

备份分为热备份和冷备份, 冷备份就是直接把文件拷走, 缺点就是需要停一下数据库。

热备份也叫逻辑备份, 不需要停数据也可以进行备份, 缺点效率低。

1.冷备份

1.停止数据库

```
[root@localhost ~]# systemctl stop mysqld
```

2.将data目录里面的所有进行压缩

```
cd /var/lib/mysql
tar -zcvf `date+%F-%H`-mysql-all.tar.gz *
```

```

[root@localhost ~]# cd /var/lib/mysql
[root@localhost mysql]# ll
总用量 122936
-rw-r-----. 1 mysql mysql      56 1月  6 18:12 auto.cnf
drwxr-x---. 2 mysql mysql      70 1月  6 19:06 heber
-rw-r-----. 1 mysql mysql     393 1月  6 19:45 ib_buffer_pool
-rw-r-----. 1 mysql mysql 12582912 1月  6 21:08 ibdata1
-rw-r-----. 1 mysql mysql 50331648 1月  6 21:08 ib_logfile0
-rw-r-----. 1 mysql mysql 50331648 1月  6 18:12 ib_logfile1
-rw-r-----. 1 mysql mysql 12582912 1月  6 19:45 ibtmp1
drwxr-x---. 2 mysql mysql      58 1月  6 21:07 mydatabase
drwxr-x---. 2 mysql mysql    4096 1月  6 18:12 mysql
-rw-r-----. 1 mysql mysql    2508 1月  6 19:11 mysql-bin.000001
-rw-r-----. 1 mysql mysql     177 1月  6 19:45 mysql-bin.000002
-rw-r-----. 1 mysql mysql    1225 1月  6 21:08 mysql-bin.000003
-rw-r-----. 1 mysql mysql      57 1月  6 19:45 mysql-bin.index
srwxrwxrwx. 1 mysql mysql      0 1月  6 19:45 mysql.sock
-rw-r-----. 1 mysql mysql      6 1月  6 19:45 mysql.sock.lock
drwxr-x---. 2 mysql mysql    8192 1月  6 18:12 performance_schema
drwxr-x---. 2 mysql mysql    8192 1月  6 18:12 sys
[root@localhost mysql]# tar -zcvf `date+%F-%H`-mysql-all.tar.gz *

```

3.备份完成后重启数据库

```

systemctl start mysqld
#可以使用scp将备份好的数据拿走，恢复直接在数据目录中解压出来

```

2.热备份

逻辑备份

1.mysqldump+binlog

mysqldump是热备份，优点：自动记录日志，可用性一致性。

```

#语法
mysqldump -h 服务器 -u用户名 -p密码 数据库名 > 备份文件.sql
#实战
mysqldump -uroot -p'admin' \
--all-databases --single-transaction \ #备份所有的库，保持数据可用性
--master-data=2 \ #注释日志记录
--flush-logs \ #刷新日志
> /backup/`date+%F-%H`-mysql-all.sql

```

2.数据还原

```

#注意数据库密码是备份文件中的密码
mysql -uroot -p'admin' < /backup/`date+%F-%H`-mysql-all.sql

```

3.binlog恢复

```

#查看最后备份时的日志文件
vim /backup/`date+%F-%H`-mysql-all.sql
#后面有多少日志文件就写几个
mysqlbinlog localhost-bin.000003 日志名 --start-position=154 |mysql -p'admin'

```

十三、mysql主从复制

1.了解主从

主从同步的作用：实时灾备，用于故障切换 读写分离，提供查询服务 备份，避免影响业务

主从复制需要用到MYSQL REPLICATION，什么是MYSQL REPLICATION？

replication可以实现将数据从一台数据库服务器（master）复制到一或多台数据库服务器（slave），默认情况下属于异步复制，无需维持长连接。通过配置，可以复制所有的库或者几个库，甚至库中的一些表（mysql - user表 - 本地权限）是MySQL内建的，本身自带的。简单的说就是master将数据库的改变写入二进制日志，slave同步这些二进制日志，并根据这些二进制日志进行数据操作

REPLICATION如何工作

(1) master将改变记录到二进制日志(binary log)中（这些记录叫做二进制日志事件，binary log events）；(2) slave将master的binary log events拷贝到它的中继日志(relay log)；(3) slave重做中继日志中的事件，修改slave上的数据。

主从复制前提

1. 主从mysql版本一致
2. 主库开启binlog日志（设置log-bin参数）
3. 主从server-id不同
4. 从库服务器能连通主库
5. 主库和从库数据一致

2.环境说明

主机名	IP地址	操作系统	角色	硬件环境
mysql-master	192.168.134.144	centos7	主库	2Core/4GMem/50Gdisk
mysql-slave	192.168.134.145	centos7	从库	2Core/4GMem/50Gdisk

3.配置hosts

```
[root@localhost ~]# vim /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.0.118 master
192.168.0.119 slave
```

4.开启二进制日志

```
vim /etc/my.cnf
log-bin=mysql-bin
server-id=1
#重启
systemctl restart mysqld
```

```
[mysqld]
port=3306
log-bin=mysql-bin
server-id=1
character_set_server=utf8
default_storage_engine=InnoDB
symbolic-links=0
max_connections=200
innodb_file_per_table=1
slow_query_log=1
long_query_time=2
datadir=/var/lib/mysql
socket=/var/lib/mysql/mysql.sock
log-error=/var/log/mysql.log
slow_query_log_file=/var/log/slowq.log
pid-file=/var/run/mysqld/mysqld.pid
user=mysql
```

5.主库授权从库

--登录数据库

```
mysql -uroot -p
grant replication slave on *.* to '授权用户'@'从库IP' identified by '授权密码';
grant replication slave on *.* to 'slave'@'192.168.134.145' identified by 'Admin123.';
flush privileges;
```

```
[root@localhost etc]# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.7.18-log MySQL Community Server (GPL)

Copyright (c) 2000, 2017, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> grant replication slave on *.* to 'slave'@'192.168.134.145' identified by 'Admin123.';
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql> 
```

6.在master查看日志文件

用于指定从库从什么位置开始进行同步，最好在备份时先执行

```
show master status\G
```

```
mysql> show master status\G;
***** 1. row *****
      File: mysql-bin.000003
      Position: 154
      Binlog_Do_DB:
      Binlog_Ignore_DB:
      Executed_Gtid_Set:
1 row in set (0.00 sec)

ERROR:
No query specified

mysql> 
```

7.备份主服务器数据

因为主从的前提是主库和从库数据一致所以需要先把主库的数据备份到从库

#备份

```
mysqldump -uroot -p'Admin123.' --all-databases --single-transaction --master-data=2 --flush-logs>`date +%F`-mysql-all.sql
```

#传输到从服务器

```
scp 2024-01-06-mysql-all.sql root@192.168.134.145:/tmp
```

```
[root@localhost ~]# ll
总用量 12
-rw-r--r--. 1 root root 1243 10月 28 2021 anaconda-ks.cfg
-rw-r--r--. 1 root root 1445 1月 6 17:28 my.cnf
-rw-r--r--. 1 root root 1043 1月 5 16:49 user_password.sql
[root@localhost ~]# mysqldump -uroot -p'Admin123.' --all-databases --single-transaction --master-data=2 --flush-logs>`date +%F`-mysql-all.sql
mysqldump: [Warning] Using a password on the command line interface can be insecure.
[root@localhost ~]# ll
总用量 772
-rw-r--r--. 1 root root 777939 1月 6 19:11 2024-01-06-mysql-all.sql
-rw-r--r--. 1 root root 1243 10月 28 2021 anaconda-ks.cfg
-rw-r--r--. 1 root root 1445 1月 6 17:28 my.cnf
-rw-r--r--. 1 root root 1043 1月 5 16:49 user_password.sql
```

8.从服务器配置

```
vim /etc/my.cnf
log-bin=mysql-bin
server-id=2
#重启
systemctl restart mysqld
```



```
[mysqld]
port=3306
log-bin=mysql-bin
server-id=2
character_set_server=utf8
default_storage_engine=InnoDB
symbolic-links=0
max_connections=200
innodb_file_per_table=1
slow_query_log=1
long_query_time=2
datadir=/var/lib/mysql
socket=/var/lib/mysql/mysql.sock
log-error=/var/log/mysqld.log
slow_query_log_file=/var/log/slowq.log
pid-file=/var/run/mysqld/mysqld.pid
user=mysql
```

9.从服务器恢复数据

为了主从数据一致，需要将master备份好的数据导入到slave

```
--临时关闭二进制日志
set sql_log_bin=0;
--恢复
source /tmp/2024-01-06-mysql-all.sql;
```

```
[root@localhost lib]# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.18-log MySQL Community Server (GPL)

Copyright (c) 2000, 2017, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> set sql_log_bin=0;
Query OK, 0 rows affected (0.00 sec)

mysql> source /tmp/2024-01-06-mysql-all.sql;
Query OK, 0 rows affected (0.00 sec)
```

10.在从服务器设置主服务器

```
--从库配置同步参数
change master to
master_host='主库IP',
master_user='授权用户',
master_password='授权密码',
master_port=3306,
master_log_file='mysql-bin.000001',    --主库日志文件
master_log_pos=453;    --position
--
change master to
master_host='192.168.134.144',
```

```
master_user='slave',
master_password='Admin123.',
master_port=3306,
master_log_file='mysql-bin.000003',
master_log_pos=154;
```

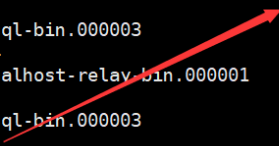
```
mysql> change master to
-> master_host='192.168.134.144',
-> master_user='slave',
-> master_password='Admin123.',
-> master_log_file='mysql-bin.000003',
-> master_log_pos=154;
Query OK, 0 rows affected, 2 warnings (0.01 sec)

mysql> █
```

11.在slave检查同步状态

```
show slave status\G
```

```
mysql> show slave status\G;
***** 1. row *****
Slave_IO_State:
  Master_Host: 192.168.134.144
  Master_User: slave
  Master_Port: 3306
  Connect_Retry: 60
  Master_Log_File: mysql-bin.000003
  Read_Master_Log_Pos: 154
  Relay_Log_File: localhost-relay-bin.000001
  Relay_Log_Pos: 4
  Relay_Master_Log_File: mysql-bin.000003
  Slave_IO_Running: No
  Slave_SQL_Running: No
  Replicate_Do_DB:
  Replicate_Ignore_DB:
  Replicate_Do_Table:
  Replicate_Ignore_Table:
  Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
```



因为没有启动所以有问题

启动主从同步

```
start slave;
```

```
mysql> start slave;
Query OK, 0 rows affected (0.00 sec)

mysql> show slave status\G
***** 1. row *****
Slave_IO_State: Waiting for master to send event
  Master_Host: 192.168.134.144
  Master_User: slave
  Master_Port: 3306
  Connect_Retry: 60
  Master_Log_File: mysql-bin.000003
  Read_Master_Log_Pos: 154
  Relay_Log_File: localhost-relay-bin.000004
  Relay_Log_Pos: 320
  Relay_Master_Log_File: mysql-bin.000003
  Slave_IO_Running: Yes
  Slave_SQL_Running: Yes
  Replicate_Do_DB:
  Replicate_Ignore_DB:
  Replicate_Do_Table:
  Replicate_Ignore_Table:
  Replicate_Wild_Do_Table:
```

